Evolutionary Neuro-Fluid Morphogenesis: A Differentiable Agent-Based Framework for Open-Ended Behavioral Adaptation in Mass-Conserving Cellular Automata

Karthik R[1,*]

[1]Department of Biotechnology (IDDD — Complex Systems and Dynamics), Indian Institute of Technology Madras, Chennai, 600036, Tamil Nadu, India

[*]Corresponding author. bs22b022@smail.iitm.ac.in

Abstract

Natural self-organising systems exhibit dynamic morphologies and adaptive behaviours that emerge purely from simple local interaction rules. Flow-Lenia, a mass-conserving continuous cellular automaton (CA), produces stable self-localising patterns (SLPs) capable of directional locomotion, yet a major limitation is that the resulting creatures often remain largely static and lack both individuality and environmental responsiveness. In this work I investigate how multicellular-like mass clusters can maintain morphological stability while evolving complex, resource-driven, open-ended behaviours such as locomotion in a simulated environment.

I present a comprehensive, differentiable framework , an Evolutionary Neuro-Fluid Morphogenesis which embeds a distributed sensorimotor brain into the existing Flow-Lenia model. As the project moved forward some of the key algorithms introduced include: (i) Multiple optimization algorithms were implemented to reduce the model's space complexity and accelerate the overall algorithm for greater ease of use; (ii) an object-oriented `OrganismTracker` that establishes agent individuality via connected-component analysis and gap-tolerance; (iii) a Torus environment with Periodic Boundary Conditions (PBC) eliminating edge artifacts via wrap-padded Sobel filters; (iv) a food-gradient environment optimised with an Evolution Strategy (ES) that achieved near-maximum theoretical mass capacity; and (v) a multi-agent ecology featuring spatially heterogeneous brains (W_{grid}) with online mutation to enable distinct organism tracking and survival. The ES-optimised agents successfully demonstrate superior foraging capabilities, efficiently moving toward and engulfing resources while preserving the strict mass conservation principles of the underlying flow lenia model.

Availability: All code files , supplementary tracking videos and other material are publicly available at `https://github.com/Karthik0042/CS6024_Project`. No external biological datasets are used.

Keywords cellular automata, Flow-Lenia, self-organisation, artificial life, neuro-evolution, open-ended evolution, JAX, mass conservation

Abbreviations CA: Cellular Automaton; SLP: Self-Localising Pattern; ES: Evolution Strategy; RT: Reintegration Tracking; MLP: Multi-Layer Perceptron; PBC: Periodic Boundary Conditions; CoM: Centre of Mass

Introduction

Nature exhibits remarkable self-organizing systems, from bacterial colonies coordinating collective behaviors to multicellular organisms developing intricate, adaptive morphologies that enable survival in dynamic environments. These emergent phenomena inspire computational models in Artificial Life (ALife), particularly Cellular Automata (CAs), which simulate life's origins through local interaction rules yielding global complexity [3]. The mathematical foundation for this kind of pattern formation was laid by Alan Turing in 1952. He showed that simple chemical reactions (called reaction-diffusion systems) could break symmetry on their own and produce complex biological patterns.[9]

This idea was later demonstrated in real chemistry by Russian scientist Boris Belousov. In his work she showed that a series of chemical reactions could oscillate on their own which helps to create regular, repeating patterns. His experiments proved patterns and rhythms can appear naturally from chaos, which was important in overturning the idea that systems inevitably decay into disorder [1].

Defining Cellular Automata

Moving forward in time from the revolutionary and groundbreaking work of Turing and Belousov, computer scientists needed a way to actually simulate these fascinating self-organizing systems mathematically. This led to the ideation and building of Cellular Automata's. A cellular automaton is essentially a grid of cells. Each

cell changes state only on the basis of its own state and nearby neighbors, using some simple fixed set of rules.[10].

In ALife research, an active focus on local interactions is essential for true emergent properties. However, debates continue on where to draw the line; systems that use global controllers or non-local communication are generally not considered true Cellular Automata [3].

Lenia advanced this approach by extending Cellular Automata into continuous space, continuous states, and continuous time, while still preserving locality through a series of convolution-based operations involving looking at the nearby region [2].

Flow-Lenia further refined the lenia model by introducing mass conservation thus preventing patterns from dissipating or proliferating without any bounds and allowing for stable Self-Localizing Patterns (SLPs) to emerge [7]. In this project, I integrate a simple distributed neural brain into the Flow-Lenia architecture without making changes to any of the known CA principles, by introducing brain weights (W_{grid}) that are spatially embedded, and sensory inputs such as food gradients or self-body contours which are derived from local Sobel convolutions, I am able to maintain locality, allowing global behaviors like foraging to arise from cellular scale local decisions.

Broader ALife Context

ALife researchers have spent many years building systems that mix simple physics with neural networks. Karl Sims's 1994 work showed how to evolve virtual creatures with bodies and brains using genetic algorithms and physics simulation, these creatures learned to walk, swim, and compete realistically [8]. In recent years, projects such as Polyworld and BioSim4 have created entire digital ecosystems where many agents live together, hunt for food, avoid predators, and evolve over generations [5, 11].

On the other hand, neural cellular automata models take a different approach in which instead of designing rules by hand, they train cells to grow specific patterns (like digits or faces) starting from random noise, using gradient descent [6]. This matches the core idea, which is to let the cells themselves learn useful behaviors. But Neural CAs mainly focus on static images and not creatures that move around or interact in changing environments.

The Flowers Group at Inria leads the Flow-Lenia based research. They started with Chan's original Lenia system [2], then added mass conservation to create Flow-Lenia [7]. Next they used evolution to make SLPs walk in specific directions [7], and most recently built AI systems to automatically discover new Flow-Lenia lifeforms [4].

Their creatures show behaviors like pulsing, gliding, even grouping into flocks, but all follow fixed rules with no ability to sense food, dangers, or other agents during simulation. This *open-loop* type of design works great for basic movement, but fails in realistic ecological environments where agents must adapt to changing resources.

Previous attempts to fix this have limitations. Evolutionary Strategies find good movement patterns but need thousands of full simulations with no real time learning [7]. Standard reinforcement learning gives agents full vision of their world, but breaks the CA locality rules by having a main controller deciding the behavior.

My approach aims to solve both these problems: In my algorithm, every cell gets its own tiny neural network with no hidden layers for simplicity that only sees its local neighborhood (using Sobel filters for edges and gradients). These per cell brains can create responses like *move toward food* or *avoid danger* that work instantly during simulation, while still keeping Flow-Lenia's physics and mass conservation intact.

Table 1 summarizes the key distinctions between various models in ALife.

Table 1 Comparison of ALife frameworks

Framework	Local Rules	Mass Cons.	Neural	Multi-Agent
Conway Game of Life	✓	–	–	–
Lenia	✓	–	–	✓
Flow-Lenia	✓	✓	–	✓
Neural CA	✓	–	✓	–
BioSim4	–	✓	✓	✓
Neuro-Fluid (Mine)	✓	✓	✓	✓

This project thus occupies a unique position, synthesizing continuous CA physics with distributed neuro-evolution to enable truly open ended, environment-coupled morphogenesis.

Problem Statement and Objectives

While Flow-Lenia successfully achieves mass conservation and stable morphology, it remains fundamentally limited as a model of open-ended Artificial Life. Specifically, I identified three critical gaps in the existing work which I aimed to address through this course project

1. **Open-Loop, Static Behaviors:** Current Flow-Lenia agents are completely ignorant and clueless about their surroundings. They follow fixed locomotion trajectories entirely dictated by their initial set of parameters, lacking any adaptive steering, sensorimotor feedback, or behavioral plasticity.
2. **Lack of Algorithmic Individuality:** Traditional CAs including the current flow lenia model treat the universe as a singular, continuous field. When a mass cluster splits or merges, the system does not recognize the distinct entities, making it impossible to track individual lifespans, lineage, or localized mutations.
3. **Absence of Ecological Dynamics:** True biological emergence requires environmental pressure. Existing continuous CAs lack interactive, resource constrained environments necessary to drive the evolutionary process to find the optimal organisms.

To bridge the gap between simple morphogenetic stability and true open-ended agency, The following set of core objectives was set :

- **Objective 1: Neuro-Fluid Morphogenesis.** I Embed a differentiable Neural Net simple *brain* directly into the continuous CA substrate. This controller processes local sensory gradients (body mass and food) via local Sobel filters and outputs real time modulations to the growth kernels which enables directed, closed-loop locomotion and also achieves greater more dynamic phenotypes.
- **Objective 2: Algorithmic Individuality.** I Developed an Object-Oriented `OrganismTracker`. Using connected-component analysis with gap-tolerance and mass thresholds, the system was able to dynamically recognize, count, and track distinct mass clusters as individual lifeforms.
- **Objective 3: Resource-Driven Optimization.** I Constructed a continuous Torus environment (utilizing wrap-padded Periodic Boundary Conditions) populated with a dynamic food field.My

aim was to optimize the neural brian controllers using an Evolution Strategy (ES), thus forcing agents to learn efficient foraging and engulfment mechanics.

- **Objective 4: Spatial Multi-Agent Coevolution (Work in Progress);** Current ongoing work focuses on scaling the framework to support a multi-agent ecology by decentralizing the neural weights into a spatial grid (W_{grid}). Although this implementation is still experimental and is undergoing active debugging, the ultimate goal is to allow distinct organisms to possess unique *genotypes* that undergo online evolutionary mutations based on real-time environmental survival, without halting the simulation.

Materials and Methods

Baseline System: Flow-Lenia

Flow-Lenia [7] improves continuous cellular automata by separating growth signals from actual movement. It evolves a continuous density field $\mathbf{A} \in [0,1]^{S_X \times S_Y \times C}$ using a two-step process: first compute growth signals, then move mass according to those signals. Unlike Lenia which adds/subtracts mass directly (breaking conservation), Flow-Lenia always preserves total mass $\iint A\,dx\,dy = \text{constant}$.

Continuous Kernels and Potentials

Each timestep processes the grid through n_k smooth convolution kernels. Every kernel K_k uses four genomic parameters (r, b, w, a) to create concentric sensing rings. These kernels scan channel $c_0[k]$ to produce potential field U_k:

$$U_k = K_k * \mathbf{A}_{c_0[k]} \tag{1}$$

This potential passes through a bell-shaped growth function with mean m_k, width s_k, and height h_k:

$$G(U_k;\, m_k, s_k, h_k) = \left[2\exp\left(-\frac{(U_k - m_k)^2}{2s_k^2}\right) - 1\right] h_k \tag{2}$$

All kernels targeting channel c sum their $G(U_k)$ outputs to create final potential U_c.

The Advection Field

Flow-Lenia converts potentials into movement using two Sobel gradients: - ∇U: pulls mass toward high-growth areas (attraction) - $\nabla \mathbf{A}_{total}$: pushes mass away from crowded areas (repulsion)

A capacity limit α prevents infinite pileup (controlled by threshold θ_A):

$$\alpha = \min\left(\left(\frac{\sum_c \mathbf{A}_c}{\theta_A}\right)^2, 1\right) \tag{3}$$

Final flow field blends attraction/repulsion based on crowding:

$$\mathbf{F} = \nabla U(1 - \alpha) - \nabla \mathbf{A}_{total}(\alpha) \tag{4}$$

Reintegration Tracking (RT)

To move mass without losing any, Flow-Lenia uses Reintegration Tracking. For each pixel at position $\mathbf{p}_{x,y}$, compute target location:

$$\mu(x, y) = \mathbf{p}_{x,y} + \text{clip}(\delta t \cdot \mathbf{F}_{x,y}, , -v_{max}, , v_{max}) \tag{5}$$

Instead of jumping across the value of the entire pixel, RT treats each pixel as a continuous square (size σ). It calculates the exact overlap areas between this square and neighboring grid pixels, distributing precise mass fractions.

This prevents floating-point errors or grid artifacts from creating or destroying mass, letting organisms move forever without dissolving.

Simulation Parameters and Configuration

To evaluate the evolutionary and neuro-fluid mechanisms,I initialized the environment with these specific configuration of hyperparameters. Table 2 outlines the core parameters that define the continuous grid, the growth kernels, the displacement limits, and the random initialization seed used throughout my experiments. Anyone reading the report can follow the same parameter set to replicate the results given in the report

Table 2 Core hyperparameters and system configuration for the Neuro-Fluid Flow-Lenia simulations.

Parameter	Value
Grid Size ($S_X \times S_Y$)	256×256
Channels (C)	2
Kernels (n_k)	5
Time Step (δt)	0.2
Capacity Threshold (θ_A)	2.0
Flow Bound (dd)	5
Flow Variance (σ)	0.65
Mixing Rule	`stoch`
Initialization Seed	777

Parameter Descriptions

- **Grid Size ($S_X \times S_Y$):** The dimensions of the continuous cellular automata landscape.
- **Channels (C):** The number of independent mass/activation fields.
- **Kernels (n_k):** The number of continuous convolution kernels governing growth and interaction rules.
- **Time Step (δt):** The integration time step used in the growth-advection solver.
- **Capacity Threshold (θ_A):** The density limit term used to prevent infinite mass accumulation via repulsion.
- **Flow Bound (dd):** The maximum distance (in pixels) that fluid mass can move in a single time step, which prevents the simulation from breaking due to overly fast flows.
- **Flow Variance (σ):** Controls the *blurriness* of the fluid flow; a higher value spreads moving mass over a wider area, acting like natural diffusion to smooth out structural boundaries.
- **Mixing Rule:** Determines how underlying parameters merge when fluids from different organisms flow into the exact same pixel. `stoch` randomly picks one source's rules based on mass proportions (preserving discrete identities), while `avg` blends them into an average, `softmax` favors the heaviest incoming mass, and `stoch_gene_wise` randomly swaps individual parameter channels like genetic crossover.
- **Initialization Seed:** The random number seed used to ensure that the initial spawn positions and the randomly generated starting physics rules are exactly reproducible.

Pipeline Optimization and Memory Efficiency

The orginal Flow-Lenia implementation relied on `jax.lax.scan` to execute simulation rollouts, which stored all T frames in GPU memory simultaneously. For extended simulations (e.g., $T = 1\,000$, $S_X = S_Y = 256$, $C = 2$, and embedded parameter maps), this $O(T)$ space complexity quickly exceeded VRAM limits, causing Out-Of-Memory (OOM) errors to come up. To resolve this and enable open-ended simulation, I implemented a series of three core memory and pipeline optimizations:

1. Single-Step JIT Compilation

Instead of Just-In-Time (JIT) compiling the entire rollout function, which forced JAX to recompile the XLA graph every time the step count T changed, I applied `jax.jit` strictly to the single `step_fn`. Because the tensor shapes and dtypes remain strictly constant across steps, the compiled block is optimally reused for every iteration with zero recompilation overhead.

Before:

```
# JIT compiles the entire loop, requiring recompilation if 'steps' changes
@partial(jax.jit, static_argnums=(2,))
def rollout(params, init_state, steps):
    # ... inner loop logic ...
```

After:

```
# JIT compiles only the physics step, completely independent of simulation length
step_fn_compiled = jax.jit(lenia.step_fn)
```

2. Memory-Efficient Streaming Rollout

I replaced the monolithic `jax.lax.scan` with a Python-level streaming loop. Instead of accumulating the entire trajectory on the device, the simulation now processes one step at a time, allowing me to stream frames directly to the CPU. This reduces peak GPU memory consumption from $O(T)$ to $O(1)$, effectively bounding memory usage to a single frame regardless of the simulation length.

Before:

```
# Allocates a massive array of shape [steps, SX, SY, C] on the GPU
return jax.lax.scan(scan_step, (init_state, params), None, length=steps)
```

After:

```
# O(1) memory: overwrites the state array in a Python loop step-by-step
for i in range(T):
    current_state = step_fn_compiled(current_state, c_params)
```

3. Asynchronous Dispatch Control

JAX utilizes asynchronous execution, meaning Python loops can queue thousands of unevaluated futures onto the GPU, eventually causing OOM crashes even if the explicit trajectory history isn't saved. I enforced a strict device synchronization using `jax.block_until_ready()` before pulling each frame to the CPU, keeping the dispatch queue flat and stable.

Before:

```
# Futures pile up in the background queue, risking OOM for large T
frame = np.array(current_state.A)
```

After:

```
# Forces GPU to finish the computation before pulling to CPU memory
frame = np.array(jax.block_until_ready(current_state.A))
```

Ultimately, these methods reduced the computational overhead and memory consumption, enabling me to run the Flow-Lenia simulations and further develop the neuro-fluid and other extensions.

Neuro-Fluid Pipeline

Instead of letting the simulation run on its own without any control, I gave each creature a way to tweak its own local rules. I call this the **protobrain algorithm**. I created a simple brain, which is just a single-layer neural network (a set of weights $\mathbf{W}$) that exists everywhere in the grid.

To make decisions, this protobrain needs to look at the creature's body. It does this by reading the gradients ($\nabla\mathbf{A}$), which I calculate using a Sobel filter. A Sobel filter is used to find edges and slopes and is used widely , it was used to tell the brain which direction the creature's mass is pointing (specifically, the horizontal and vertical slopes).

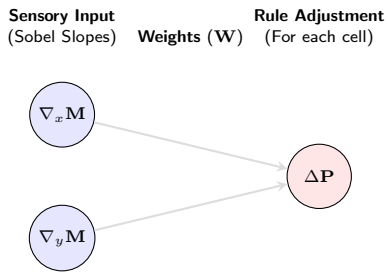


Figure 1 The Protobrain Architecture: This acts as the actual point-wise neural network for the agents. For simplicity, the output is shown as a single node ($\Delta\mathbf{P}$), which represents the unique parameter adjustments calculated independently for every single cell in the grid based on its local spatial slopes.

Once the brain sees these slopes, it decides how to change the base rules ($\mathbf{P}_{\text{base}}$). I calculate the new, adjusted rules ($\mathbf{P}_{\text{pheno}}$) using this simple formula:

$$\mathbf{P}_{\text{pheno}}(x,y) = \text{clip}\left(\mathbf{P}_{\text{base}}(x,y) + \alpha \tanh\left(\sum_{i=1}^{2C} \nabla\mathbf{A}_i(x,y)\mathbf{W}_i\right),\ 0,\ 1\right), \tag{6}$$

Here, $\alpha = 0.15$ is just a small limit I set so the brain can only make small adjustments. This keeps the creature from instantly blowing up or falling apart.

Algorithm 1 The Protobrain Algorithm

Require: The creature's body $\mathbf{A}$, the base rules $\mathbf{P}_{\text{base}}$, and the brain weights $\mathbf{W}$

$\mathbf{M} \leftarrow \sum_{c=1}^{C} \mathbf{A}_c$ ▷ Step 1: Calculate the total mass of the creature

$\nabla\mathbf{M} \leftarrow \mathbf{M} \circledast \mathbf{K}_{\text{Sobel}}$ ▷ Step 2: Find the horizontal and vertical slopes using the Sobel filter

$\mathbf{P}_{\text{pheno}} \leftarrow \text{clip}\left(\mathbf{P}_{\text{base}} + \alpha \tanh(\nabla\mathbf{M} \cdot \mathbf{W}),\ 0,\ 1\right)$ ▷ Step 3: Let the brain tweak the rules based on the slopes

$\mathbf{U} \leftarrow \mathcal{G}(\mathbf{A} \circledast \mathbf{K}_{\text{Lenia}};\ \mathbf{P}_{\text{pheno}})$ ▷ Step 4: Calculate where the creature should grow or shrink

$\mathcal{F} \leftarrow \nabla\mathbf{U} \odot (1 - \mathbf{A}^2) - \nabla\mathbf{M} \odot \mathbf{A}^2$ ▷ Step 5: Figure out which way the fluid should flow

$\mathbf{A}_{t+1} \leftarrow \text{Reintegration}(\mathbf{A}, \mathcal{F})$ ▷ Step 6: Move the fluid to its new position

return $\mathbf{A}_{t+1}, \mathbf{P}_{\text{base}}$ ▷ Return the updated body and keep the base rules safe

To teach this protobrain how to actually move across the screen, I needed a way to optimize its weights. Standard neural network training (backpropagation) takes up far too much memory for a long simulation, so I went with a simple guess-and-check method called an Evolution Strategy.

My goal was locomotion. To track movement, I calculated the creature's Center of Mass (CoM) at the end of the run:

$$c_x = \frac{\sum x \cdot \mathbf{A}(x,y)}{\sum \mathbf{A}(x,y)}, \quad c_y = \frac{\sum y \cdot \mathbf{A}(x,y)}{\sum \mathbf{A}(x,y)} \tag{7}$$

The fitness score is just the straight-line distance it traveled from its starting point. If it falls apart and loses too much mass, it gets a score of zero.

Algorithm 2 Evolution Strategy for Locomotion

Require: Initial random brain $\mathbf{W}^*$, noise scale σ, number of generations G

Best Fitness $f^* \leftarrow \mathcal{J}(\mathbf{W}^*)$ ▷ Test the initial brain

for $g = 1 \ldots G$ **do**

  $\mathbf{W}_{\text{mutant}} \leftarrow \mathbf{W}^* + \sigma\mathcal{N}(0, \mathbf{I})$ ▷ Create a mutant brain by adding random noise

  Run the simulation with $\mathbf{W}_{\text{mutant}}$ for T steps

  Calculate the new distance traveled, f_{mutant}

  if $f_{\text{mutant}} > f^*$ **then** ▷ If the mutant went further than our best so far

    $\mathbf{W}^* \leftarrow \mathbf{W}_{\text{mutant}}$ ▷ The mutant becomes the new best brain

    $f^* \leftarrow f_{\text{mutant}}$

  end if

end for

return $\mathbf{W}^*$ ▷ Return the smartest brain

Individuality and the Torus World

To actually build a multi-agent ecosystem, I needed a way to tell where one creature ends and another begins. Because Flow-Lenia is fundamentally a continuous soup of fluid, I treat organism tracking as an image-processing problem.

First, I look at the grid's total mass ($\mathbf{M}$) and create a binary mask $\mathbf{B}$ by cutting off anything below a threshold τ. Because the fluid advection can sometimes stretch a creature and create tiny internal gaps, I use a morphological "closing" operation with a small square filter (S_{gap}) to stitch those gaps back together.

Algorithm 3 Binary Mapping Morphological Tracking (scipy.ndimage)

Require: Frame $\mathbf{M}$, threshold τ, structuring element S_{gap}

$\mathbf{B} \leftarrow \mathbf{M} > \tau$ ▷ Step 1: Threshold the continuous fluid into a strict binary mask

$\mathbf{M}_{\text{closed}} \leftarrow (\mathbf{B} \oplus S_{gap}) \ominus S_{gap}$ ▷ Step 2: Dilate then erode to visually stitch tiny gaps together

labels, $N_{\text{features}} \leftarrow \text{connected_components}(\mathbf{M}_{\text{closed}})$ ▷ Step 3: Group adjacent active pixels into distinct IDs

return labels, $N_{extfeatures}$

After bridging the gaps, I evaluate the connected blobs. If a blob's total mass is too small, I throw it out as background noise.

Algorithm 4 General Organism Tracker

Require: Frame $\mathbf{M}$, min_mass ρ_{min}

labels, $N \leftarrow$ Algorithm 3($\mathbf{M}$) ▷ Get initial candidate blobs

valid_organisms $\leftarrow \emptyset$

for $i = 1 \ldots N$ **do** ▷ Iterate through every detected blob

  $\text{mass}_i \leftarrow \sum \mathbf{M}[\text{labels} == i]$ ▷ Calculate the actual fluid mass belonging to this blob

  if $\text{mass}_i \geq \rho_{min}$ **then** ▷ Filter out tiny fragments or dissipating noise

    valid_organisms $\leftarrow$ valid_organisms $\cup \{i\}$

  end if

end for

return valid_organisms

I also had to fix the world's boundaries. The original simulation had hard walls (zero-padding), which caused the fluid to smash into the edges and dissipate. To fix this, I changed the world into a Torus (like a continuous donut). I replaced the standard Sobel edge-detector with a periodic version that wraps around the screen.

Algorithm 5 Periodic Boundary Conditions (Torus Sobel)

Require: Grid $\mathbf{A}$, Sobel kernels $\mathbf{K}_x, \mathbf{K}_y$

$\mathbf{A}_{pad} \leftarrow \text{pad}(\mathbf{A}, \text{mode} = \text{'wrap'}, \text{width} = 1)$ ▷ Wrap the edges so the top touches the bottom, left touches right

$\nabla_x\mathbf{A} \leftarrow \text{convolve2d}(\mathbf{A}_{pad}, \mathbf{K}_x, \text{mode} = \text{'valid'})$ ▷ Compute X gradients smoothly across boundaries

$\nabla_y\mathbf{A} \leftarrow \text{convolve2d}(\mathbf{A}_{pad}, \mathbf{K}_y, \text{mode} = \text{'valid'})$ ▷ Compute Y gradients smoothly across boundaries

return $\nabla_x\mathbf{A}, \nabla_y\mathbf{A}$

Finding Food and Evolving to Forage

We know that a creature that just moves in a straight line isn't very smart. I wanted them to actively move with a purpose. So, I introduced an external food field $\mathbf{F}$. Food is inserted into the world as distinct circular patches of high concentration that contains flow lenia mass.

Organisms can *eat* this food if they become big enough and sit on top of it. The digestion math absorbs the food state directly into the agent's mass state:

$$\text{Eaten}_{x,y} = \gamma\left(\mathbf{F}_{x,y} \odot I(\mathbf{M}_{x,y} > \rho)\right) \tag{8}$$

$$\mathbf{A}_{t+1} = \mathbf{A}_t + \frac{\text{Eaten}}{C}, \quad \mathbf{F}_{t+1} = \mathbf{F}_t - \text{Eaten} \tag{9}$$

Here, γ is the digestion rate, and $I(\cdot)$ is an indicator function that evaluates to 1 if the local mass $\mathbf{M}_{x,y}$ exceeds the threshold ρ, and 0 otherwise. This ensures they have enough localized mass to actually consume the patch.

To help them find this food, the protobrain was upgraded to a 4-input neural network. It now reads the x and y slopes of both its own body ($\nabla\mathbf{A}$) and the food field ($\nabla\mathbf{F}$).

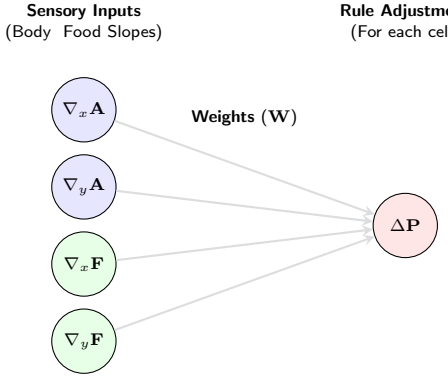


Figure 2 The Upgraded Protobrain Architecture: The point-wise neural network now processes 4 spatial inputs—the gradients of the organism's own body mass alongside the gradients of the external food field. The output $\Delta\mathbf{P}$ dictates the local behavioral parameter shifts.

Algorithm 6 4-Input Foraging Brain Forward Pass

Require: Body mass $\mathbf{A}$, Food field $\mathbf{F}$, Brain Weights $\mathbf{W}$
$\nabla_x\mathbf{A}, \nabla_y\mathbf{A} \leftarrow$ Algorithm 5($\mathbf{A}$) ▷ Extract body gradients
$\nabla_x\mathbf{F}, \nabla_y\mathbf{F} \leftarrow$ Algorithm 5($\mathbf{F}$) ▷ Extract food gradients (smell)
$\mathbf{v}_{sense} \leftarrow$ concatenate($[\nabla_x\mathbf{A}, \nabla_y\mathbf{A}, \nabla_x\mathbf{F}, \nabla_y\mathbf{F}]$, axis $= -1$) ▷ Combine into 4-channel input
$\Delta\mathbf{P} \leftarrow \tanh(\mathbf{v}_{sense} \cdot \mathbf{W}) \times 0.15$ ▷ Calculate parameter shifts via neural network
$\mathbf{P}_{pheno} \leftarrow$ clip($\mathbf{P}_{base} + \Delta\mathbf{P}, 0, 1$) ▷ Apply shifts and keep them bounded
return $\mathbf{P}_{pheno}$

To teach this brain to forage, I used an Evolution Strategy (ES) loop. Interestingly, I realized I didn't need a complex mathematical reward. Since eating directly adds mass to the organism's body, and the Flow-Lenia physics naturally causes unstable shapes to dissipate, my objective function $\mathcal{J}$ is incredibly simple: it is just the total mass of the creature at the end of the simulation run (at time T).

$$\mathcal{J}(\mathbf{W}) = \sum_{x,y} \mathbf{A}_T(x,y) \tag{10}$$

If the creature successfully navigates to the food patches and eats, it grows. If it wanders aimlessly or its body falls apart, its final mass is tiny. The ES algorithm simply tries to maximize this final mass.

Algorithm 7 Evolution Strategy for Foraging Brains

Require: Initial weights $\mathbf{W}^*$, population size n, noise variance σ, generations G, steps T
for $g = 1 \dots G$ **do**
  for $i = 1 \dots n$ **do** ▷ Evaluate candidate population
    $\epsilon_i \sim \mathcal{N}(0, \mathbf{I})$
    $\mathbf{W}_i \leftarrow \mathbf{W}^* + \sigma\epsilon_i$ ▷ Add Gaussian noise to create a mutant brain
    $\mathbf{A}_T \leftarrow$ Simulate($\mathbf{W}_i$, steps $= T$) ▷ Run physics with mutant brain
    $f_i \leftarrow \sum \mathbf{A}_T$ ▷ Fitness is just the total final mass!
  end for
  $i_{\text{best}} \leftarrow \arg\max_i f_i$ ▷ Find the brain that ate the most
  if $f_{i_{\text{best}}} > \mathcal{J}(\mathbf{W}^*)$ **then**
    $\mathbf{W}^* \leftarrow \mathbf{W}_{i_{\text{best}}}$ ▷ Update the global best brain
  end if
end for
return $\mathbf{W}^*$

Multi-Agent Ecology and Spatial Brains

Finally, I wanted multiple creatures interacting on the same grid. To do this, I replaced the single brain with a massive spatial brain tensor: $\mathbf{W}_{\text{grid}}$. Every single pixel on the grid carries its own distinct set of neural weights that physically move alongside the fluid mass.

Instead of training offline, I implemented an online spatial evolution mechanism. Every 100 steps, I use the tracking algorithm to identify distinct creatures. I copy the spatial brain weights from the most massive, well-fed organisms, add mutation, and overwrite the neural parameters of the dying pieces. This creates localized survival of the fittest scenario directly inside the continuous soup.

Implementation Details

I wrote the entire pipeline in JAX to take advantage of GPU acceleration. To keep things fast, I compiled the main physics and brain updates a single time using `jax.jit`. Memory was a major bottleneck, so rather than storing thousands of simulation frames in memory at once, I used a streaming approach that processes and encodes frames directly to video using MoviePy and FFMPEG at 60 fps. Organism tracking is handled by `scipy.ndimage`. Everything runs entirely in Python with no external biological datasets required for the run.

Module-Specific Hyperparameters

With the core substrate configured, I introduced the *Protobrain* to achieve directed locomotion. Additional parameters are required to scale the neural modulation and track the distinct organism entities correctly using my connected-components algorithm.

Table 3 Hyperparameters for the Protobrain locomotion and organism tracking modules.

Parameter	Value
Brain Modulation Scalar	0.15
Tracker Activation Threshold	0.2
Tracker Gap Tolerance (pixels)	5
Tracker Minimum Mass	10.0

For the foraging simulations, I introduced stationary food patches into the environment. The organism's brain must integrate this new

sensory information, and the physics engine is updated to handle the digestion of environmental mass.

Table 4 Hyperparameters governing the Food Environment, Sensory Integration, and Digestion mechanics.

Parameter	Value
Food Sensory Multiplier	5.0
Digestion Rate (γ)	0.1
Eating Mass Threshold (ρ)	0.8
Initial Food Patches	12
Food Patch Radius	8

To optimize the neural weights for the foraging task, I employed an Evolution Strategy (ES). The optimization loop evaluates batches of organisms over extended time horizons to gradient-ascend toward maximum mass accumulation.

Table 5 Hyperparameters for the Evolution Strategy (ES) optimization algorithm.

Parameter	Value
Population Size (N)	16
Noise Standard Deviation (σ_{ES})	0.1
Learning Rate (α)	0.05
Evaluation Horizon (steps)	300
Maximum Generations	20

Finally, in the open-ended Multi-Agent Ecology simulation, I adjusted the grid size to maintain computational tractability while simulating many interacting agents. Brains are spatially embedded and updated online, requiring specific decay and noise parameters for the transmission of traits to off-shooting masses.

Table 6 Hyperparameters specific to the Multi-Agent Ecology and spatial online evolution.

Parameter	Value
Ecology Grid Size ($S_X \times S_Y$)	128×128
Initial Food Patches	80
Brain Inheritance Decay Rate	0.95
Online Mutation Noise (σ_{online})	0.02
Brain Mask Dilation (iterations)	12
Tracker Minimum Mass	20.0

Results and Discussion

Pipeline Optimization and Memory Efficiency

Three issues were identified and fixed in the original pipeline and the results are shown below .

The first was memory. Using `jax.lax.scan`, space complexity was $\mathcal{O}(T \times SX \times SY \times C)$. After the fix, this dropped to $\mathcal{O}(SX \times SY \times C)$. Time complexity remained $\mathcal{O}(T \times SX \times SY \times K)$ in both cases.

The second was JIT recompilation. The original compile cost was $\mathcal{O}(U \times t_{\text{xla}})$ where U is the number of unique T values ever used. After the fix, compilation happens once at cost $\mathcal{O}(1 \times t_{\text{xla}})$. Runtime per step stays $\mathcal{O}(SX \times SY \times K)$ in both cases.

The third was GPU syncing. The original left $\mathcal{O}(T)$ pending operations queued before any executed. After the fix, the queue depth is always $\mathcal{O}(1)$.

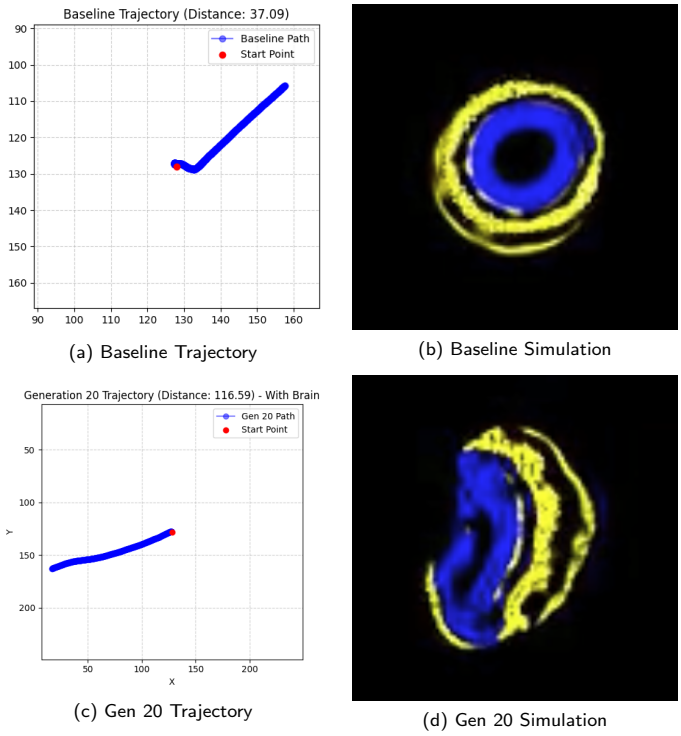
Together these reduced peak GPU memory significantly:

Table 7 Peak GPU memory before and after optimization.

Configuration	**Peak Memory (Before → After)**
Flow Lenia (256×256)	$\mathcal{O}(T) \rightarrow \mathcal{O}(1)$ per frame
Flow Lenia (64×64, $T = 10^4$)	655 MB → ∼64 KB
Flow Lenia + Params (256×256)	3.0 GB → ∼2.0 MB

A 500-step run of flow Lenia finishes in 0.10 seconds. This freed up enough memory to make the Evolution Strategy and Multi-Agent experiments feasible.

Proto Brain and Directional Locomotion



(a) Baseline Trajectory

(b) Baseline Simulation

(c) Gen 20 Trajectory

(d) Gen 20 Simulation

Figure 3 Baseline (a,b) vs Generation 20 (c,d).

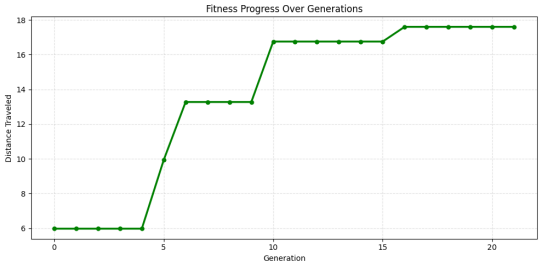


Figure 4 Locomotion evolution trends across generations.

Figure 3 illustrates how the shape of the organism changes to optimize locomotion. From a biological perspective, we can observe a distinct split in the activation channels, representing a break in symmetry that emerges to drive the organism forward. Additionally, Figure 4 shows the evolution trends of locomotion, showing how the organism steadily optimized its brain parameters over successive generations to maximize movement and distance traveled.

Organism Tracking and Periodic Boundary Conditions (PBC)

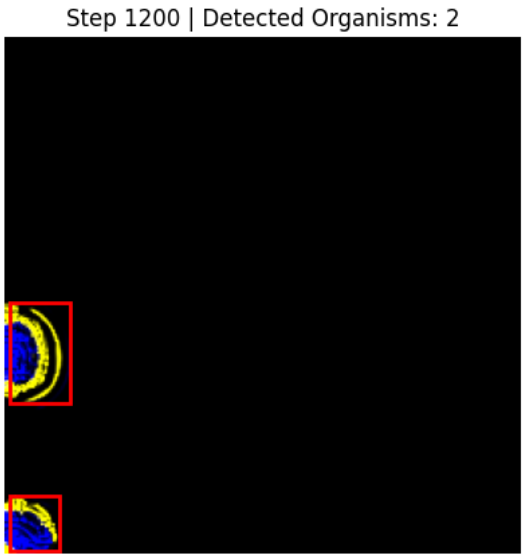
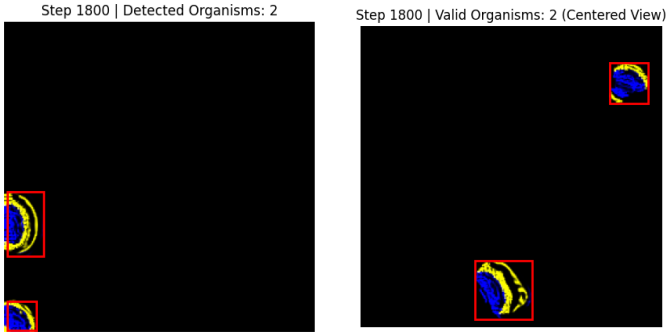


Figure 5 Organism tracking demonstration with bounding box.

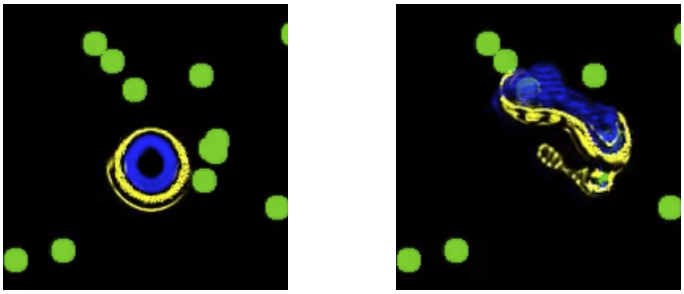


(a) Before PBC (b) After PBC

Figure 6 Comparison of organism behavior before (a) and after (b) implementing Periodic Boundary Conditions.

As seen Figure 5 I added a tracking system using bounding boxes and connected component labeling to follow individual organisms by their mass . This lets me accurately track each organism's mass, movement, and behavior separately. I also switched to Periodic Boundary Conditions (PBC) to make the environment infinite. As shown in Figure 6, during the intial runs the organisms used to get damaged or lose mass when hitting the edges. Now they wrap around smoothly to the other side. This is important for long-term evolution and natural movement.

Foraging Optimization



(a) Organism before simulation (b) Organism after optimization

Figure 7 Comparison of organism behavior before (a) and after (b) optimizing for foraging

As shown in Figure 7, the evolutionary algorithm optimizes the organism's localized spatial brain (W_{grid}) to better detect and follow food gradients. This allows the agents to actively search for and absorb mass. Interestingly, during this process, a complex behavior emerges: the organism spontaneously splits into multiple separate entities. This lets it gather food from different places at the same time, greatly improving its feeding efficiency. All of this happened naturally just from the pressure to maximize food intake.

Towards Open-Ended Evolution : Dynamical Ecology

In the final stage, I moved from isolated optimization tasks to a full dynamical ecological environment designed for open-ended evolution. I created an environment with consumable food that forced organisms to actively forage to grow and survive. By spatially distributing neural weights on the grid (W_{grid}), each organism carries and updates its own localized brain. Combined with the continuous tracking system, this enables real-time spatial evolution in which organisms can now mutate and improve their foraging strategies directly inside the simulation. This removes the need for any external genetic algorithm and supports truly autonomous open-ended artificial life.

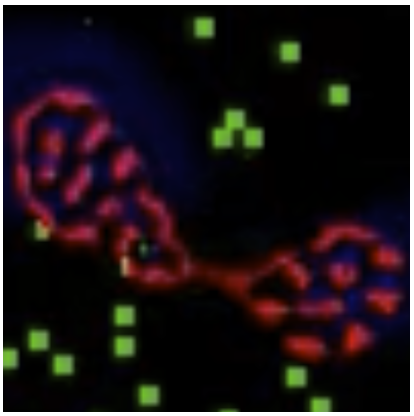


Figure 8 Organism learning to undergo mitosis

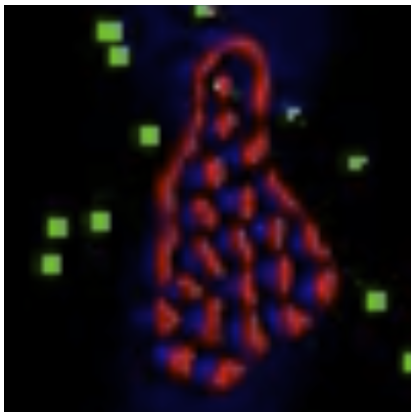


Figure 9 Organism learning to make a pseudopodia to absorb food

Figures 8 and 9 show complex behaviors similar to those of real amoeba. Thanks to continuous state space, our artificial organisms can change shape in very flexible ways. Their localized brains (W_{grid}) respond directly to nearby food and mass gradients. When an organism gathers enough mass through foraging, it can split into two , just like amoeba reproduction or cell division.

Limitations and Discussion

While this framework successfully demonstrates the core objectives, there are a few limitations to take note of that i came across. The biggest obstacle I faced was starting the simulation with multiple organisms and creating a stable system. Spawning multiple organisms at the start often led to destabilization of the simulation, likely because overlapping flow vectors confuse the sensory inputs of their protobrains. As a result, multi-agent ecologies had to emerge naturally through spontaneous splitting rather than manual placement.

Additionally, the dynamic spatial evolution parameter space model using ($\mathbf{W}_{\text{grid}}$) is experimental. Because evolutionary updates happen in a periodic Python loop every 100 steps, it is not really continuous. The fixed radius masks used to transfer *brains* during splitting can also occasionally flow down into empty space causing future organisms not to have the brain parameters.

However, on a broader level, these constraints highlight the nature of emergent complexity. Behaviors like spontaneous amoeboid splitting and pseudopodia formation were never explicitly mentioned when establishing the rules. They emerged purely from the back and forth of mass conservation, simple rules defined within a simple brain, and environmental food pressure. This demonstrates how even basic physics substrates can create complex, life-like survival strategies, a central theme in Artificial Life research.

Conclusion

This project evolved from a simple study of cellular automata into a pipeline for open-ended artificial life. By introducing a spatial neural brain, periodic boundary conditions, organism tracking, and a dynamic food environment on top of the existing Flow-Lenia algorithm, I created a rich and evolving ecological environment.

The key results show that evolutionary optimization increased directed locomotion (distance traveled increased from 37 to 116+ units in a span of just 20 generations) and maximized foraging efficiency. notably, streaming memory optimizations made long simulations practical, reducing peak memory from 3 GB to just 2 MB.

In general, towards the end of the project, the most satisfying result was witnessing unexpected emergent phenomena, such as spontaneous cell division and morphological adaptation. It proves that this continuous, fluid framework is a highly capable sandbox for exploring how rudimentary physical rules can naturally give rise to adaptive, living behaviors.

GitHub Link

The complete code, all the iteration notebooks, and supplementary simulation videos/images are available in the following github repository:

`https://github.com/Karthik0042/CS6024_Project`

Acknowledgements

I would like to thank Prof. Manikandan Narayanan for giving me the opportunity to implement my ideas through this project and for providing valuable feedback after both the proposal and the intermediate presentation. I also acknowledge the FLOWERS group for scheduling a call with me. Their insights greatly helped me understand the Lenia codebase, which provided the foundational implementation of Cellular Automata from which this work departs. The BioSim4 project [5] inspired the design philosophy of neural control. Compute resources were provided by Google Colaboratory (A100 and T4 GPU instances).

References

1. Boris P. Belousov. A periodic reaction and its mechanism. *Collection of Abstracts on Radiation Medicine*, 147:145, 1959.
2. Bert Wang-Chak Chan. Lenia: Biology of artificial life. *Complex Systems*, 28(3):251–286, 2019.
3. Christopher G Langton. Studying artificial life with cellular automata. *Physica D: Nonlinear Phenomena*, 22(1):120–149, 1986. Proceedings of the Fifth Annual International Conference.
4. Thomas Michel, Marko Cvjetko, Gautier Hamon, Pierre-Yves Oudeyer, and Clément Moulin-Frier. Exploring flow-lenia universes with a curiosity-driven ai scientist: Discovering diverse ecosystem dynamics. *Proceedings of the Artificial Life Conference 2025*, pages 633–643, 2025.
5. David R. Miller. Biosim4: An open-source simulator for evolving artificial life. GitHub repository, 2022.
6. Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020.
7. Quentin Plantec et al. Flow-lenia: Towards open-ended evolution in cellular automata through mass conservation and localisation. *arXiv preprint arXiv:2212.07906*, 2023.
8. Karl Sims. Evolving virtual creatures. *Proceedings of SIGGRAPH '94*, pages 15–22, 1994.

9. Alan Mathison Turing. The chemical basis of morphogenesis. *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences*, 237(641):37–72, 1952.
10. John von Neumann. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966.
11. Larry Yaeger. Computational genetics, physiology, metabolism, neural systems, learning, vision, and behavior in polyworld. *Artificial Life*, 1994.